AutoDoc AI — Automotive Architecture Test Document

Document Type: Synthetic Educational Example
Purpose: Testing document indexing and Retrieval-Augmented Generation (RAG)

Note: This is a fictional educational example, not an official AUTOSAR specification. Do not use it for production engineering or safety decisions.

1. Introduction

An automotive software system contains multiple software components that work together to provide vehicle functions. A layered architecture separates responsibilities and helps developers understand how these components communicate.

2. Architecture Layers

For this simplified example, the software platform has three conceptual layers.

2.1 Application Layer

The Application Layer contains software components responsible for vehicle functions. For example, a component named `VehicleStatusManager` processes vehicle status information.

2.2 Runtime Environment (RTE)

The Runtime Environment (RTE) provides communication mechanisms between application software components and lower-level software services. It helps components interact through defined interfaces.

2.3 Basic Software (BSW)

Basic Software provides common software services and hardware-related functionality. It supports the application software by providing services needed by the overall platform.

3. Example of Data Flow

Consider a fictional automotive system with two software components:

- **VehicleStatusManager:** Receives a `VehicleSpeed` signal and checks whether the value is available.

- **DashboardDisplay:** Receives a status update and displays the relevant information to the driver.

In this example, the RTE supports communication between `VehicleStatusManager` and `DashboardDisplay`.

The document does not specify how the vehicle calculates its speed or how the dashboard physically displays the information.

4. Benefits of Layered Architecture

A layered architecture can provide the following benefits:

1. **Separation of responsibilities:** Each layer has a defined role.
2. **Modularity:** Developers can work on individual software components.
3. **Reusability:** Components may be reused in different applications when their interfaces and requirements are compatible.
4. **Maintainability:** Clear interfaces can make software easier to update and maintain.

5. Information Not Specified

This example does not provide the following information:

- Maximum vehicle speed.
- Automotive Safety Integrity Level (ASIL) classification.
- Exact memory usage of the RTE.
- The name of a real vehicle manufacturer.
- The production vehicle models using this architecture.

If a question asks for any of these details, the answer should state that the information is not available in this document rather than guessing.

6. Conclusion

This simplified example demonstrates how an automotive software architecture can organize application components, communication mechanisms, and basic software services into conceptual layers. It is suitable for testing document retrieval, evidence-based question answering, and the ability of a RAG system to identify missing information.